EDLAB: A Benchmark for Edge Deep Learning Accelerators

Hao Kong and Shuo Huai
School of Computer Science and Engineering,
Nanyang Technological University,
Singapore 639798
HP-NTU Digital Manufacturing Corporate Lab,
Nanyang Technological University,
Singapore 637460

Di Liu and Lei Zhang
HP-NTU Digital Manufacturing Corporate Lab,
Nanyang Technological University,
Singapore 637460

Hui Chen
School of Computer Science and Engineering,
Nanyang Technological University,
Singapore 639798
HP-NTU Digital Manufacturing Corporate Lab,
Nanyang Technological University,
Singapore 637460

Shien Zhu
School of Computer Science and Engineering,
Nanyang Technological University,
Singapore 639798

Shiqing Li and Weichen Liu
School of Computer Science and Engineering,
Nanyang Technological University,
Singapore 639798
HP-NTU Digital Manufacturing Corporate Lab,
Nanyang Technological University,
Singapore 637460

Manu Rastogi
HP Labs, HP Inc., Palo Alto, CA 94304 USA

Ravi Subramaniam
Innovations and Experiences – Business Personal Systems, HP Inc., Palo Alto, CA 94304 USA

Madhu Athreya and M. Anthony Lewis
HP Labs, HP Inc., Palo Alto, CA 94304 USA

Editor's notes:
This article introduces a new end-to-end benchmark to evaluate the performance of edge deep learning accelerators.
—Sai Manoj, George Mason University

Deep learning (DL) algorithms or deep neural networks (DNNs) have achieved considerable success in recent years, demonstrating outstanding performance in image classification [1], [2], machine translation [3], etc. To pursue higher accuracy, DL models are growing *wider* and *deeper*, i.e., more complex. Therefore, DL models are usually trained and deployed on powerful servers.

However, executing DL models on cloud or remote servers suffers from two critical issues: *latency issue*—some safety-critical systems, such as self-driving cars and autonomous Unmanned Aerial Vehicles (UAVs), are subject to rigorous latency requirements, but inferring results from remote servers is difficult to guarantee latency requirements due to network bandwidth and availability; *privacy issue*—some data are confidential and the data owner is unwilling to upload these data to remote cloud and servers. This drives the birth of *edge computing* [4], a computing service close to users acting like a middle-ware between cloud and end-users.

Digital Object Identifier 10.1109/MDAT.2021.3095215
Date of publication: 6 July 2021; date of current version: 22 April 2022.

Copublished by the IEEE CEDA, IEEE CASS, IEEE SSCS, and TTTC

Edge systems are usually constrained by power, memory, cost, and physical limitations [4]. Therefore, the power-hungry hardware, like high-end GPUs with more than 100 W power consumption, may not be deployed in some cases with limited power supply. However, computation-intensive DL models require powerful computing units to provide sufficient computation for on-time/real-time operation. Thus, it results in a *computational gap*. To close the computational gap, edge DL accelerators (EDLAs) draw increasing attention from research and industry. Various low-power and efficient EDLAs are designed to speed up the inference in different edge environments.

The emerging EDLAs feature diversely underlying hardware characteristics and employ various software tools for edge DL implementation and adoption. The hardware and toolchain diversity of EDLAs makes it difficult to directly and efficiently compare the performance and cost of different EDLAs and in turn prohibits DL practitioners to effectively and efficiently deploy their newly designed DL models on a suitable EDLA. Moreover, the state-of-the-art benchmark methods, like [5]–[7], focus on conventional performance metrics' competition like highest accuracy, or lowest latency/highest throughput, which are inadequate for making a thorough evaluation for EDLAs and lack an evaluation from hardware perspective.

EDLAs have a wide spectrum from high-performance computing unit installed within communication stations to low-power, less-capable wearables. Simply knowing the best accuracy or latency for a DNN model on diverse EDLAs cannot provide insightful information for system designers, especially when developing an in-house edge intelligence system. It is unable to answer the practical question as follows:

> Can we further improve a developed model on an EDLA for better accuracy without compromising performance?

Table 1 shows an experimental result of Wide-ResNet [8] with different width configurations on a representative edge device, NVIDIA Jetson Xavier, where width scaling factor means the number of channels is scaled up by that value. We see that the model benefits from the increasing width with higher accuracy, while not influencing the latency. For EDLAs with low or mid computational capability, since they are likely to be resource-limited, it is important to know the performance upper bound of EDLAs and DL models' efficiency, analogous to Roofline model [9] for applications on multicore systems, such that designers can exploit the full potential of EDLAs to achieve the best tradeoff between accuracy and performance.

Table 1. Accuracy and latency of Wide-ResNet model with different width scaling on NVIDIA Jetson Xavier.

Model	Scaling factor	Accuracy(%)	Latency(ms)
Wide-ResNet	1	88.09	5.5
	2	92.71	5.58
	4	94.19	5.58

However, evaluating hardware performance bound of EDLAs exposes a great challenge. Different from CPUs which usually have several performance counters to provide system information to evaluate applications' efficiency, EDLAs are like a "*black-box*" system, because they lack performance counters to monitor their operational status. Therefore, it needs a *systematic* method to approximately evaluate EDLAs' performance bound.

In this article, we propose a new benchmark method to evaluate EDLAs in a more comprehensive way, dubbed EDLA benchmark (EDLAB). EDLAB has two unique features: 1) it provides a tool to integrate different software tool-chains into a unified framework such that it can conveniently deploy DNN models onto various EDLAs and 2) we develop a *parameterized model* (PM) to evaluate the performance bound of black-box-like EDLAs. By means of PM, EDLAB is capable of efficiently evaluating the execution efficiency of different DNN models on EDLAs. The proposed EDLAB is an easy-to-use tool for machine learning practitioners who wish to quickly prototype their newly proposed DNN algorithms on edge systems in the coming edge era. Although some EDLAs are developed for DL training, most EDLAs are devised for DNN inference. The current version of EDLAB tends to only benchmark the inference EDLAs. We may extend EDLAB for training as our future work.

EDGE DL accelerators

In this section, we use three widely used EDLAs as examples to present the challenges of benchmarking EDLAs, such as the heterogeneity of different

EDLAs. Then, we demonstrate how these difficulties are addressed in EDLAB.

- *Edge Tensor Processing Unit (TPU)*: Google develops Edge TPU to complement Cloud TPU to provide fast inference at the network edge, where TPU is built based on the core component of systolic array processors. To deliver high performance at a low resource cost, Edge TPU employs INT8 quantization to compress DNN models to reduce the memory footprint. For the deployment software, it relies on TensorFlow Lite to quantize DNN models and compile models into executable workloads.
- *Neural compute stick (NCS)*: Intel NCS is designed as a plug-in gadget to be integrated into some legacy systems which do not have AI boosting capability, where the core component of NCS is an array of very long instruction word (VLIW) vector processors. NCS uses OpenVINO, a vendor-provided DL library, to convert the models trained in other frameworks like TensorFlow to the format supported by NCS, where the model is quantized into FP16. It is worth noting that NCS is only applicable for AI vision tasks.
- *Jetson Xavier*: Jetson Xavier is a powerful edge GPU platform developed by Nvidia. Since its GPU architecture is compatible with Nvidia's Desktop GPU, the most of existing DL frameworks can be directly employed. However, to achieve a better performance, Nvidia provides TensorRT framework to optimize the DNN inference by quantizing the model and fusing tensors.

From the above three EDLAs, it is not difficult to see that EDLAs differ from each other in terms of the underlying hardware architecture and the compiler used for deployment. The underlying architecture of EDLAs is decisive for the performance of EDLAs, which is fixed and cannot be modified during the deployment. Besides the architecture, the compiler or model optimizer provided by vendors is also of great significance for improving the actual performance of EDLAs. These compilers are configurable, and different configurations can lead to diverse performance improvements, which makes it challenging to fairly evaluate and compare different EDLAs. For example, TensorRT provides multiple quantization precision, including INT8, FP16, and FP32, which bring different latency-accuracy tradeoffs. To solve this problem, on the one hand, EDLAB unifies the deployment configuration to preserve the consistency of the workload when comparing different EDLAs. On the other hand, EDLAB integrates different optimization methods provided by the toolchain, which enables to explore the best performance of EDLAs under different deployment options.

Benchmark methodology

This section introduces the design philosophy of EDLAB, including the benchmarking models ("Benchmarking models" section), benchmark workload processing ("Workload processing framework" section), and selected metrics ("Benchmark metrics" section). Figure 1 presents the overview of EDLAB. First, DL applications and PMs are designed and trained in general DL frameworks. Then these trained models are sent to EDLAB, which determines the deployment of hyperparameters (e.g., inference batch size, quantization bitwidth, etc.) according to the target platform and the metric to evaluate and convert the models into various intermediate representations for different EDLAs. The converted deployable workload is deployed to the corresponding EDLA to perform inference and measure the proposed metrics. Finally, the obtained performance data of multiple metrics will be post-processed and analyzed by EDLAB in a fine-grained way.

Benchmarking models

In EDLAB, we prepare two categories of DNN models for comprehensively benchmarking EDLAs with regard to different metrics, which will be discussed in detail in the "Benchmark metrics" section.

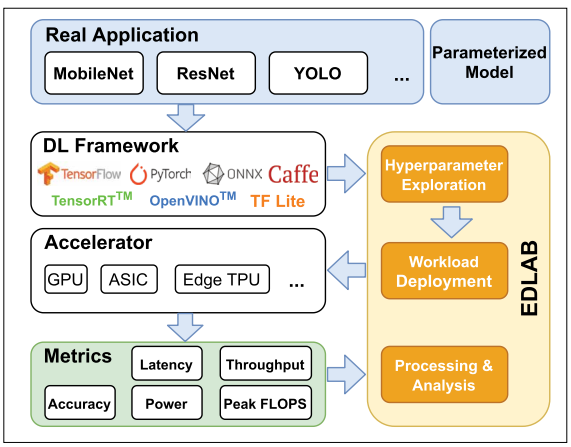

Figure 1. Overview of EDLAB.

The state-of-the-art DNN models: Since in many DL applications, DNN models are developed based on these state-of-the-art (SOTA) models by using *transfer learning*, selecting some representative DNN models as the partial benchmark suite is a must. For the current EDLAB, we mainly target EDLAs with resource and power constraints, so we carefully selected several distinct DNN models as shown in Table 2, which can be well supported by the EDLAs we have evaluated.[1] Natural language processing (NLP) models are not involved here because they are too large for some resource-constraint EDLAs. Moreover, some operators of NLP models are not well supported by some accelerators, such as Intel NCS, which is only applicable for vision tasks. In addition, DNN design is gradually shifting from manual design to automatic design, thus we take MnasNet [10] designed by neural architecture search (NAS) to follow the development trend. However, it is worth noting that there is no any limitation in selecting the SOTA models for evaluation in EDLAB. Users are free to select the desired models. We have prepared API to seamlessly add new models into EDLAB.

Parameterized model: Only using SOTA models suffers from a flaw—we can only report some straightforward metrics but cannot evaluate EDLA's performance bound which is critical for resource-constrained EDLAs, because hardware performance bound can provide practical information for designers to optimize their model to sufficiently utilize precious hardware resources. Unfortunately, as explained in the introductory part, this aspect has been missed in previous DNN benchmark tools.

To model the performance bound of a target EDLA, we need to collect adequate operational information. However, most of EDLAs do not provide related tools to monitor its internal status during run time. This motivates us to design a PM to evaluate the hardware performance under different configurations and model hardware performance bound in a "*black-box*" fashion.

Figure 2 demonstrates our PM design, which contains a convolutional layer, a batch normalization layer, and a rectified linear unit (ReLU) activation layer. This combination is the most commonly used basic block in modern DNN design and accounts for the majority of computational overhead of DNNs. Therefore, we can customize the basic block to mimic different DNNs. There are six adjustable architecture parameters in total, which constitute a huge design space. By analyzing many DNNs, we design a reasonable range for each parameter to tune, which is summarized in Table 3. To make EDLAB applicable for EDLAs equipped with different computing capability and provide guides for designing more resource-efficient network architectures, the tuning ranges are designed to be slightly larger than existing DNN design space. Since it is prohibitive to involve all combinations of these parameters, we randomly sample 3,000 combinations to approximately model the hardware performance upper bound.

[1] More models were evaluated, but we encountered some model conversion issues for some EDLAs. To have a fair comparison, we only keep these four for the current EDLAB and plan to investigate more in the next version.

Table 2. SOTA DNN models. Classification models are trained on ImageNet [11] and the detection model is trained on COCO [12] data set.

Model	MACs(G)	Params(M)	Top-1/mAP	Description
Inception_V3	5.73	27.16	78.0	Classification
MobileNet_V2	0.3	3.47	71.8	Classification
MnasNet_B1	0.32	3.9	74.5	Classification
SSD-MobileNet_V2	0.3	3.47	24.0	Detection

Workload processing framework

The successful adoption of EDLAs relies on general DL development frameworks, such as TensorFlow, PyTorch, MXNet, as well as vendor-provided software tool sets. The general DL frameworks are

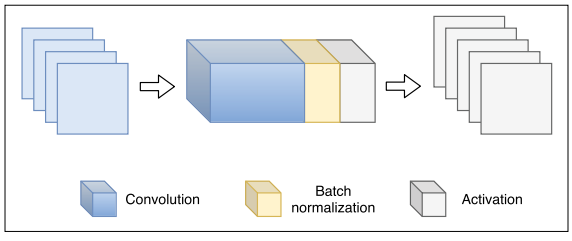

Figure 2. Architecture of the PM.

Table 3. Tuning range of the PM.

Parameter	Range
Resolution	[8, 10, 12, ..., 448]
Input planes	[16, 18, 20, ..., 2048]
Output planes	[16, 18, 20, ..., 2048]
Kernel size	[1, 3, 5, 7]
Stride	[1, 2]
Groups*	[1, 2, 4, ..., 2048]

*The maximum number of groups cannot exceed the number of input planes in a convolutional layer.

employed to design and train models while vendor-provided tools optimize the trained models and convert the models into platform-specific intermediate representation.

Different platform-specific tools are not compatible with each other, which may lead to the inconsistency of converted workload for different platforms and consequently affect the fairness of the benchmark results. In the proposed model conversion framework shown in Figure 3, we integrate different model conversion tools and uniformly control the model conversion parameters in the upper layer, which ensures that the workload generated for different platforms remains consistent. In the current version, the framework takes as input a trained model from TensorFlow, determines the conversion hyperparameters according to the target platform and the metric to evaluate, and call the corresponding vendor-provided tool to perform the conversion. For now, there are three adjustable hyperparameters: 1) *inference batch size*; 2) *input resolution*; and 3) *quantization precision*. The batch size depends on the metrics to evaluate, which will be discussed in detail in the "Benchmark metrics" section. The input resolution is determined by the benchmark data set and the model, and the quantization precision is related to the hardware platform. With these information, EDLAB is capable of converting the model from TensorFlow into platform-specific deployable files.

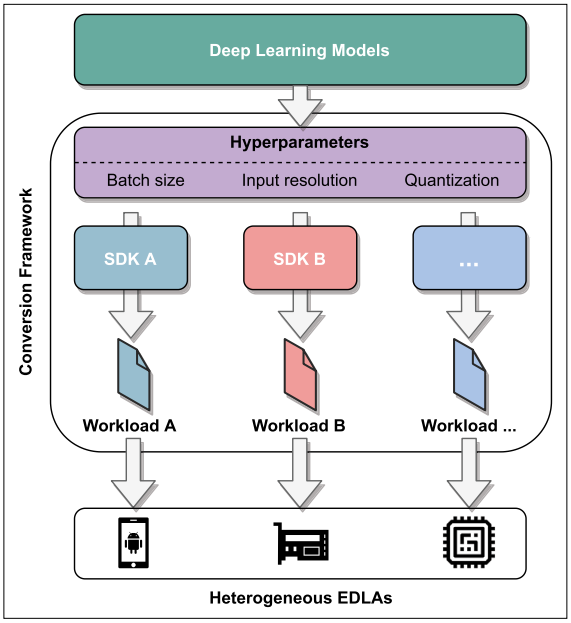

Figure 3. Unified model conversion framework, which converts a DL model into platform-specific deployable workload.

Benchmark metrics

The design goal of EDLAB is to provide a comprehensive evaluation for EDLAs which can cover various design concerns of edge systems. Therefore, in EDLAB, we evaluate and report several metrics. To accurately measure each metric, EDLAB separately customizes the deployment and benchmark settings for each metric.

Accuracy (*ACC*): Usually, the accuracy can be reproduced once the model structure is determined and weights are well trained and fixed. However, EDLAs employ different tools to convert pre-trained models for efficient execution and such conversion may change the model structures, data representation, and bit width, hence it may cause drop in accuracy. It is important to report this accuracy drop for accuracy-sensitive applications. EDLAB calculates the accuracy from the prediction results of all images in the test data set. Each image will be predicted only once to mitigate the benchmarking cost. For simplicity, the *batch size* of inference is set to 1.

Latency (*L*): Latency indicates the time elapsed between one input data and its corresponding prediction. For some systems with rigorously temporal requirement, e.g., autonomous driving, latency is essentially critical. In EDLAB, we only measure the latency of the given model processing one image, thus the *batch size* has to be 1 for the evaluation of latency. To avoid the variance caused by different system status, EDLAB runs a model on one image by 50 times and calculates the average value as the model's latency.

Throughput (*T*): The throughput of an EDLA is the largest number of samples processed within 1 s. The throughput is considered as an important metric for some applications, such as video analytic, which relies on the high-throughput performance of hardware to guarantee high frame-per-second (FPS) requirement. For both image classification and object detection, the throughput unit is *images/second.* To measure the highest throughput of EDLAs, We make the *batch size* as large as possible until the memory runs out. Similar to the latency measurement, each input batch will be repeated 50 times.

FLOPS (*F*): We also measure the number of floating-point operations executed per second (FLOPS)

to evaluate the execution efficiency of EDLAs. FLOPS can help evaluate the execution efficiency of different EDLAs under various DNN models. FLOPS is derived from the throughput and the number of operations which a DNN model has, as shown in the following equation:

$$F = T \times \text{FLOPs} \quad (1)$$

where T is the throughput of a DNN model on an EDLA and FLOPs denote the number of floating-point operations the DNN model has in total. Note that FLOP(S) indicates the hardware performance, while FLOP(s) represents the DNN model's complexity.

Power (*P*): Some edge intelligence systems, especially those supplied by batteries, are subject to limited power budget and prefer to achieve an expected performance under limited power budget. However, power measurement relies on the power sensor to accurately obtain it. For those EDLAs which have internal on-board sensors, like Nvidia Jetson series, we can directly obtain power consumption from power sensors. For those without on-board sensors, power has to be measured by an external power meter. It is worth noting that the power consumption of an EDLA varies under different running configurations, and EDLAB only reports the highest power consumption, which can be obtained under the benchmark configuration of the throughput measurement.

Efficiency (*E*): The efficiency of EDLAs is a combined and more comprehensive metric, which is calculated as

$$E = \frac{T}{P} \quad (2)$$

where T is the throughput and P is the power consumption of the accelerator when achieving that throughput. The unit of *efficiency* is *images per second per watt* (imgs/s/W). This metric unifies performance and power into one combined metric, which is instrumental when comparing various EDLAs with different computational capability and power consumption levels.

Evaluation

In this section, we use EDLAB to benchmark three off-the-shelf EDLAs, analyze the benchmarking results, and provide some insightful observations.

Table 4. Hardware platforms.

Name	Precision	Software
NCS2+Raspberry Pi4	FP16	OpenVINO
Edge TPU	INT8	TFlite
Jetson Xavier	INT8,FP16,FP32	TensorRT

Experimental settings

The test data of classification models and the object detection model are from ImageNet2012 validation set and COCO2017 validation set, respectively. Before evaluating each metric, EDLAB first executes the model for ten iterations for hardware warm-up. More detailed experimental settings for each metric are presented in the "Benchmark metrics" section.

Evaluation devices

The three EDLAs are listed in Table 4. Since quantization is a promising technique to reduce the model size and boost model inference, some EDLAs support different quantization bit-width configurations, from INT8 to FP32. For the EDLAs which support different quantization bit-width configurations, we evaluate all quantization configurations supported. NCS2 is a plug-in gadget which needs to be combined with one host machine. In our setting, we combine it with Raspberry Pi4 to set up an intact edge system. Raspberry Pi4 is responsible for processing data, while NCS2 is responsible for conducting DNN inference.

Results and analysis

All the benchmarking results are reported in Table 5, including *accuracy, latency, throughput, power*, and *conversion time* which denotes the time to convert a general model to the format that EDLAs support. In addition, we visualize *throughput, FLOPS,* and *efficiency* in Figure 4. Since Jetson Xavier can directly support general frameworks like TensorFlow, etc., we show the benchmarking results of Xavier without using the optimization tool, TensorRT (denoted as woRT).

Accuracy: For all data bitwidth, Xavier performs better in prediction accuracy than other accelerators that use the same data bitwidth. For instance, Xavier (INT8) outperforms Edge TPU by 0.08%–0.5% accuracy advantage. Also, for Xavier (FP16) and NCS2, there is an accuracy gap ranging from 0.4% to 2.55%. This reveals that different quantization strategies and internal data representation affect the

Table 5. Experimental results of basic metrics.

Model	Accelerator	Latency (ms)	Accuracy (%)	Throughput (imgs/s)	Power (W)	Conversion Time (ms)
Inception_V3	NCS2 (FP16)	83.13	75.24	21.44	6.1	17990
	Edge TPU (INT8)	52.77	77.62	18.95	**4.69**	**17455.98**
	Xavier (INT8)	**3.55**	77.7	**641.73**	8.01	1579809.94
	Xavier (FP16)	4.9	77.79	428.84	15.12	589188.488
	Xavier (FP32)	14.51	77.82	111.06	21.84	42808.4085
	Xavier (woRT)	27.21	**77.83**	101.70	20.67	0
MobileNet_V2	NCS2 (FP16)	25.08	70.72	86.6	5.61	**6960**
	Edge TPU (INT8)	2.56	70.18	384.62	**5.08**	N.A*
	Xavier (INT8)	1.45	70.68	**1670.76**	6.49	325319.1373
	Xavier (FP16)	**1.43**	71.16	1657.69	7.04	237770.8
	Xavier (FP32)	2.57	71.15	810.73	10.47	24038.95
	Xavier (woRT)	6.91	**71.17**	309.84	10.89	0
MnasNet_B1	NCS2 (FP16)	33.56	73.13	63.35	5.49	9690
	Edge TPU (INT8)	3.5	70.94	285.71	**4.89**	**6051.2**
	Xavier (INT8)	**1.73**	71.26	**1194.59**	7.82	275228.83
	Xavier (FP16)	1.75	73.53	1193.49	8.56	222774.5
	Xavier (FP32)	3.17	73.52	614.26	12.02	24841.12
	Xavier (woRT)	8.14	**73.54**	265.40	15.13	0
SSD-MobileNet_V2	NCS2 (FP16)	70.13	23.6	27.86	5.96	44830
	Edge TPU (INT8)	**15.45**	21.7	**64.72**	4.71	N.A.*
	Xavier (woRT)	47.26	**24**	34.02	**4.02**	0

*MobileNet_V2 and SSD-MobileNet_V2 for Edge TPU are TensorFlow Lite version directly taken from Google.

prediction quality. For accuracy-sensitive applications, it should be taken into account.

Latency: From the results of Xavier, the quantized models can achieve up to four latency reduction, but interesting to see that lower bit-width quantization does not necessarily lead to lower latency. For example, Xavier (INT8) has higher latency than Xavier (FP16) for MobileNet_V2. The latency reduction of Edge TPU from Inception_V3 to MobileNet_V2 and MnasNet_B1 is much more significant than NCS2. This may be because these two models are developed by Google and they develop these models by considering underlying hardware architecture. This somehow indicates that hardware-–software co-design method can better exploit the limited resources of EDLAs.

Throughput: Figure 4a shows the throughput results of EDLAs. All EDLAs achieve higher throughput for MobileNet_V2 and MnasNet_B1 than for Inception_V3, since MobileNet_V2 and MnasNet_B1 are designed for mobile devices with fewer operations than Inception_V3, which seeks for higher accuracy without considering hardware limitation. Edge TPU attains 20.3× throughput improvement for MobileNet_V2 compared to Inception_V3, where the corresponding throughput improvement of NCS2 and Xavier (FP32) are 4.04× and 7.30×, respectively. This again confirms the importance of hardware–software co-design. In addition, model optimizations adopted by EDLAs also bring throughput improvement. For instance, Xavier (INT8) has 5.35× higher throughput than Xavier (woRT). Even using the same data precision, Xavier (FP32) achieves higher throughput than Xavier (woRT).

Power: The high computing capability of Xavier comes at the cost of higher power consumption. An interesting finding about the power consumption is that quantization contributes to reducing the

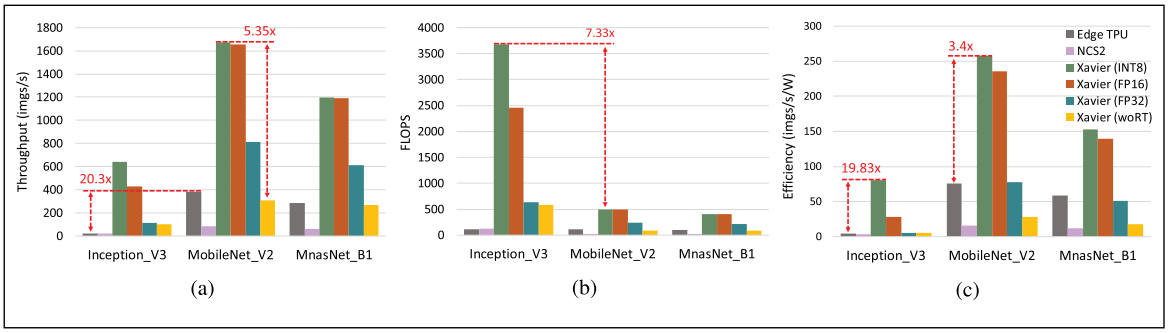

Figure 4. Results of derived metrics. (a) Throughput. (b) FLOPS. (c) Efficiency.

power consumption. Xavier (INT8) achieves up to 2.73× power reduction for Inception_V3 compared to Xavier (FP32). In addition, the large model consumes more power than the two mobile setting models, and we think it is because large models need more resources to exploit the model's parallelism.

FLOPS: Figure 4b is the results of FLOPS of EDLAs for the three DNN models. In contrast to the throughput results, all EDLAs achieve the highest FLOPS performance for Inception_V3, even though MobileNet_V2 and MnasNet_B1 have higher throughput. The gap of FLOPS for different models on the same platform is up to 7.35×, because large DNN models have more parallel operations, which can employ more computing cores to work concurrently and thus process more operations at the same time. This inspires the need of PM proposed in the "Workload processing framework" section.

Efficiency: From Figure 4c, since Edge TPU consumes less power, the efficiency difference between Xavier (INT8) and Edge TPU is reduced. For the example of MnasNet, if we only look at the throughput between Xavier (FP32) and Edge TPU, Xavier (FP32) has better throughput. However, in terms of efficiency metric, Edge TPU is better and this indicates that Edge TPU utilizes power more efficiently.

Model conversion cost: EDLAB strives to provide an end-to-end test for all tested EDLAs. Table 5 shows the conversion cost for the three EDLAs. The majority of this time cost is from the conversion tools provided by vendors. We can see that the conversion cost is not negligible. Since automated DNN design which explores many hyperparameter configurations to find a best design is becoming the mainstream, testing different designs on the target EDLAs would be a significantly timing-consuming procedure. This suggests to model the hardware at the system level, such that the design cost can be reduced. We envision PMs may be used for this purpose.

PM benchmarking

In this section, we show the benchmarking results of using PMs of EDLAB. Due to the space limitation, we only show the results conducted on Nvidia Jetson Xavier. The experimental results are shown in Figure 5, where the blue dots indicate the PMs. It is clear to see that by means of PMs, EDLAB can identify the hardware upper bound of Xavier under different complexity in terms of latency (Figure 5a) and FLOPS (Figure 5b). We place four existing models in the performance and latency spectrum. Except the three used in our previous evaluation, we add VGG16 to highlight the efficiency issue.

We can see that no model achieves the best performance of the tested EDLA. Especially, the old VGG16 has the largest number of FLOPs but the lowest efficiency (100× difference between the performance bound and the model performance), thereby leading to the longest latency. The execution efficiency has been improved for the recently developed models, but still they have not achieved the best performance on the hardware.

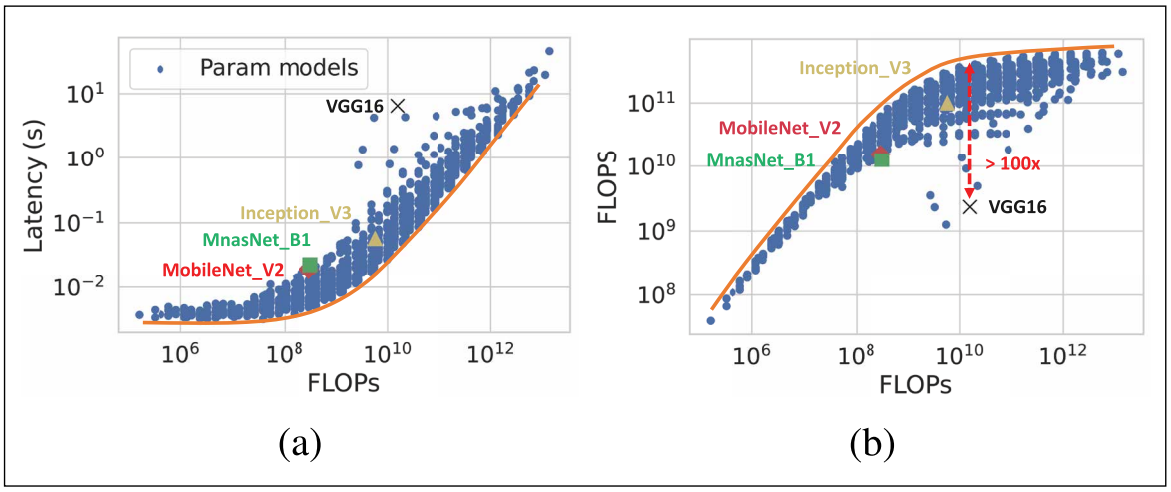

Figure 5. Results of PM. (a) Latency. (b) Performance.

As we know that for DNN models, *the deeper and wider, the better*. This implies that we should bear in mind the underlying hardware when designing edge intelligence systems.

> *Observation*: Although the latest DNN models are more efficient, hardware-agnostic DNN designs cannot fully utilize the resources provided by EDLAs.

To address the challenges of benchmarking EDLAs, we present a unified benchmark methodology and a benchmark tool EDLAB. EDLAB integrates multiple software development kits (SDKs) and unifies the workload processing and deployment for different EDLAs. One novelty of EDLAB is to integrate PMs which enable us to model the performance bound of EDLAs and quickly evaluate the performance of diverse DNN models on different EDLAS. We use EDLAB to benchmark three off-the-shelf EDLAs and the experimental results show the applicability of EDLAB and provide some interesting observations which may help the design of efficient DNN models. In future, we will keep expanding EDLAB to integrate more types of DL tasks and support more emerging EDLAs. In addition, EDLAB can be combined with NAS, model compression, and model scaling to design effective and efficient edge intelligence systems. ■

Acknowledgments

This study was supported under the RIE2020 Industry Alignment Fund – Industry Collaboration Projects (IAF-ICP) Funding Initiative, as well as cash and in-kind contribution from the industry partner, HP Inc. This work was also supported in part by NTU NAP M4082282 and SUG M4082087, Singapore.

References

[1] K. He et al., "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[2] C. Szegedy et al., "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2818–2826.

[3] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*. [Online]. Available: http://arxiv.org/abs/1810.04805

[4] W. Shi et al., "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.

[5] V. J. Reddi et al., "MLPerf inference benchmark," 2019, *arXiv:1911.02549*. [Online]. Available: http://arxiv.org/abs/1911.02549

[6] A. Ignatov et al., "Ai benchmark: Running deep neural networks on Android smartphones," in *Proc. Eur. Conf. Comput. Vis. (ECCV) Workshops*, Sep. 2018, pp. 288–314.

[7] C. Coleman et al., "Dawnbench: An end-to-end deep learning benchmark and competition," in *Proc. ML Syst. Workshop, Co-Located 31st Conf. Neural Inf. Process. Syst. (NIPS)*, 2017, p. 102.

[8] S. Zagoruyko and N. Komodakis, "Wide residual networks," 2016, *arXiv:1605.07146*. [Online]. Available: http://arxiv.org/abs/1605.07146

[9] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM,* vol. 52, no. 4, pp. 65–76, 2009.

[10] M. Tan et al., "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 2820–2828.

[11] J. Deng et al., "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.

[12] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. IEEE Conf. Eur. Conf. Comput. Vis. (ECCV)*. Cham, Switzerland: Springer, 2014, pp. 740–755.

Hao Kong is pursuing a PhD with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. His main research interests include adaptive deep neural network design and deep learning acceleration. Kong has a bachelor's degree from the University of Electronic Science and Technology of China (UESTC), Chengdu, China.

Shuo Huai is pursuing a PhD with Nanyang Technological University, Singapore. His research interests include edge computing, hardware and software co-design, and machine learning acceleration. Huai has a BS from Shandong University, Qingdao, China.

Di Liu is a Postdoctoral Researcher at HP-NTU Digital Manufacturing Corporate Lab, Nanyang

Technological University, Singapore. His research interests include hardware-aware deep neural network design. Liu has a Ph.D. from Leidon University, Leiden, The Netherlands.

Lei Zhang is a Research Associate at HP-NTU Digital Manufacturing Corporate Lab, Nanyang Technological University, Singapore. Her research interests include hardware and software co-design, deep neural network compression, and application of machine learning techniques in edge computing. Zhang has an MS from Chongqing University, Chongqing, China.

Hui Chen is pursuing a PhD with Nanyang Technological University, Singapore. Her current research interests include embedded and real-time systems, multi-core scheduling, and network-on-chip. Chen has a BE from Zhejiang University, Hangzhou, China, and an MSc from the National University of Singapore, Singapore.

Shien Zhu is pursuing a PhD with Nanyang Technological University, Singapore. His research interests include neural network quantization and model compression. Zhu has a BE from the University of Science and Technology of China (USTC), Chengdu, China.

Shiqing Li is pursuing a PhD with Nanyang Technological University. His current research interests include FPGAs, sparse matrices operations, and high-level synthesis (HLS). Li has a BS and an MS in computer science and technology from Shandong University, Jinan, China.

Weichen Liu is a Nanyang Assistant Professor at the School of Computer Science and Engineering, Nanyang Technological University, Singapore. His research interests include embedded and real-time systems and machine learning acceleration. Liu has a PhD from The Hong Kong University of Science and Technology, Hong Kong.

Manu Rastogi is a Machine Learning Scientist at HP Labs, HP Inc., Palo Alto, CA, USA. His research interests include machine learning algorithms and their implementations. Rastogi has a PhD from the University of Florida, Gainesville, FL, USA.

Ravi Subramaniam is a Distinguished Technologist at HP Inc., Palo Alto, CA, USA. His research interests include the Internet of Things (IOT) and wearable computing. Subramaniam has a PhD from the State University of New York at Buffalo, Buffalo, NY, USA.

Madhu Athreya is a Distinguished Technologist at HP Labs, HP Inc., Palo Alto, CA, USA. His research interests include computer vision and multimedia. Athreya has an MBA from Cornell University, Ithaca, NY, USA.

M. Anthony Lewis is the Vice President of HP Labs, HP Inc., Palo Alto, CA, USA. His research interests include biorobotics, neuromorphic engineering, and machine learning. Lewis has a PhD from the University of Southern California, Los Angeles, CA, USA.

■ Direct questions and comments about this article to Weichen Liu, School of Computer Science and Engineering, Nanyang Technological University, Singapore 639798, and HP-NTU Digital Manufacturing Corporate Lab, Nanyang Technological University, Singapore 637460; wchliu@gmail.com.